

Solo Harness — Rubric Talking Points

What to **say** and what to **show** for each requirement

Rishik Kolpekwar
Gauntlet AI · 24-hour Build Challenge

DON'T TRIP ON THIS

One **worker = Claude** (the brain the harness governs). The **scraper is MATERIAL HANDLING** — a pillar, not a worker. Your **"second worker" is a different brain** (Claude Opus, or a mock worker), **not** the scraper. Say it this way and you hit material handling AND the swap bonus.

MUST

required to pass

1 Four pillars, separate from the worker

- SAY** the harness wraps ONE worker (Claude). the four pillars are separate, named modules — I can point to each file: `tools/` (material handling), `guardrails.ts`, checkpoints, alarms.
- SHOW** architecture diagram: worker in the center, four labeled boxes around it. then the actual code tree.

2 Agent behavior changes from pillar feedback

- SAY** the worker changes what it does based on the pillars: a low-relevance item is dropped before it ever sees it; a failed hallucination checkpoint makes it strip the uncited claim and re-run.
- SHOW** before/after: an item scored `0.31` dropped by the threshold; a checkpoint FAIL forcing a re-run of just that stage.

3 Guardrails declared · checkpoints explicit pass/fail

- SAY** guardrails aren't buried in a prompt — they're declared constants. every checkpoint has an explicit pass/fail rule.
- SHOW** the constants on screen: `RELEVANCE_THRESHOLD = 0.4`, `NO_SEND_WITHOUT_APPROVAL`, token cap, allow-lists. a stage stamped pass / fail.

4 Structured alarms

- SAY** every alarm is a structured object — named type, severity, context, recommended action. not a log string.
- SHOW** `{ type: "GUARDRAIL_VIOLATION", severity: "critical", context: "send without approval", recommendedAction: "hard block + log" }`

5 Runs on real input at demo time

- SAY** this runs on MY real work — today's actual brief from my news, jobs, and inbox, scored against my profile: pathology AI, AI infra, VC, quant.
- SHOW** the live Solo UI generating today's brief, end to end.

6 HARNESS.md exists

- SAY** architecture and all four pillars are documented in HARNESS.md in the repo.
- SHOW** scroll the file — the four pillar headings + the architecture diagram.

SHOULD

strong credit

7 Swappable agent interface

- SAY** the harness only depends on a `Worker` interface. no pillar references Claude or the SDK. dropping in a different agent needs zero harness changes.
- SHOW** the `Worker` interface and the `makeWorker()` factory — the loop calls `worker.step()`, nothing model-specific.

8 Checkpoints persisted · replayable

- SAY** every checkpoint persists to SQLite. I can replay from checkpoint 3 without re-running stages 1 and 2.
- SHOW** kill a run mid-way, then replay from a checkpoint — prior stages are not recomputed.

9 Human-in-the-loop escalation

- SAY** the harness stops and asks instead of guessing: outbound drafts hit a review gate, and HIGH / CRITICAL alarms pause the run for acknowledgment.
- SHOW** the review gate (approve / edit / reject) before anything sends; an alarm pausing the run.

10 Second worker swapped in live

SAY same harness, same four pillars, new brain: I swap Claude **Sonnet** → **Opus** (and show a tiny `MockWorker`) with zero code changed in any pillar.

SHOW the worker name flips in the logs from `claude:sonnet-4-6` to `claude:opus-4-8` — same checkpoints and alarms still fire.

One-line close: "four pillars, separate and named · the worker is swappable behind one interface · and it ran on my real data, live." · map the scraper to **material handling**, never to "worker."